

Unità di esecuzione in Linux

Processi e pipe

Roberto Marino, PhD

Università di Messina
CdL in Informatica — Reti e Sistemi Distribuiti Mod.B

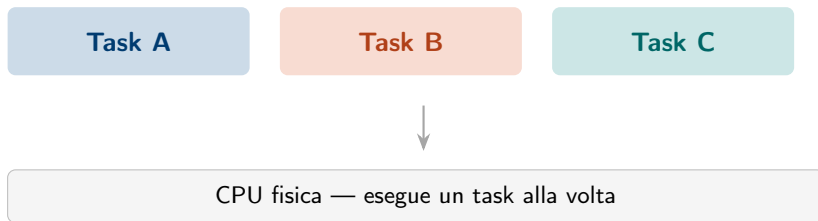
29 aprile 2026



UniMe
1548

Un sistema operativo moderno deve mandare avanti decine di attività contemporaneamente: un editor, un browser, demoni di rete, processi di sistema. Ma la CPU esegue istruzioni *una alla volta*. Come si concilia questo?

La risposta è l'astrazione di **unità di esecuzione** e uno *scheduler* che le alterna così velocemente da sembrare simultanee.

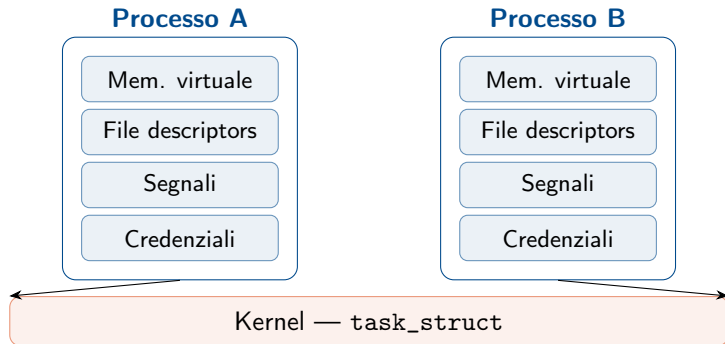


“The Linux kernel does not separate threads and processes; to the kernel, everything is just a schedulable entity called a task.”

— Robert Love, *Linux Kernel Development*, 3rd ed.

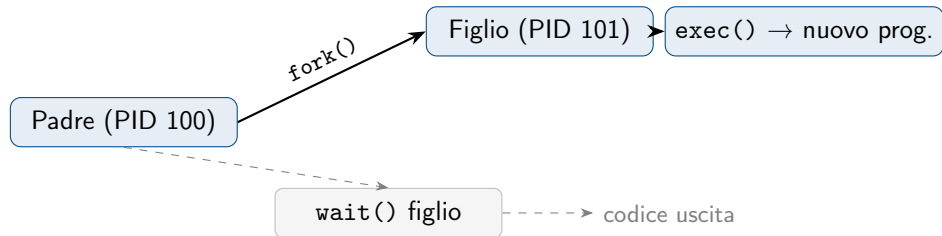
Cos'è un processo

Un **processo** è la principale unità di *isolamento* del kernel. Ogni processo ha un proprio spazio di indirizzamento virtuale, un insieme di file aperti, credenziali, e una o più unità di esecuzione (thread) al suo interno.



fork(): duplicare un processo

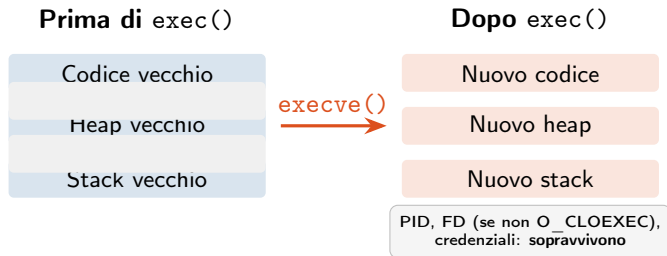
fork() è la syscall che crea un nuovo processo come copia esatta del chiamante. Padre e figlio condividono inizialmente le stesse pagine fisiche (*copy-on-write*): la copia effettiva avviene solo alla prima scrittura.



Il pattern **fork** + **exec** è quello che esegue la shell ogni volta che lanci un comando: la shell fa `fork()`, il figlio trasforma se stesso con `exec()`, il padre aspetta con `wait()`.

`exec()`: trasformarsi in un altro programma

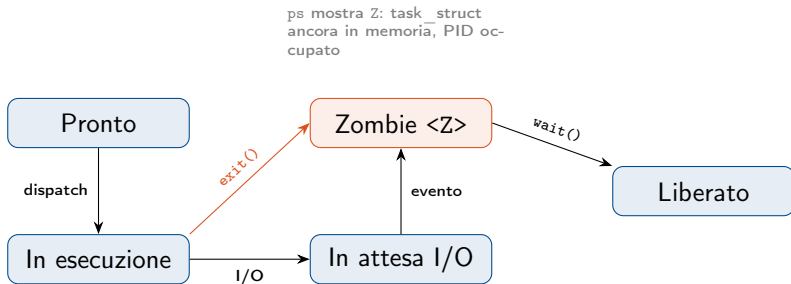
`execve()` sostituisce completamente l'immagine del processo: codice, heap e stack vengono rimpiazzati. Il PID rimane lo stesso.



Cosa **non** sopravvive: handler dei segnali, memoria condivisa, lock. Cosa **sopravvive**: PID, PPID, file descriptor aperti (a meno di `O_CLOEXEC`), credenziali, directory corrente.

`wait()`: raccogliere i figli — e lo zombie

Quando un processo termina con `exit()`, la sua `task_struct` non viene subito liberata: il kernel la mantiene finché il padre non raccoglie il codice di uscita con `wait()` o `waitpid()`. Fino ad allora il processo è uno **zombie**.



`waitpid(pid, &status, WNOHANG)` è la versione non bloccante: ritorna subito se il figlio non è ancora terminato, utile nei server che gestiscono molti figli.

Ogni processo porta con sé un insieme ricco di attributi. I più importanti:

- ▶ **PID / PPID** — identificatore e padre
- ▶ **UID / GID** (reale ed effettivo) — credenziali
- ▶ **CWD** — directory di lavoro corrente
- ▶ **Tabella dei file descriptor** — tutti gli fd aperti
- ▶ **Maschera dei segnali** — quali segnali sono bloccati
- ▶ **Limiti** (rlimit) — max fd, max memoria, CPU time

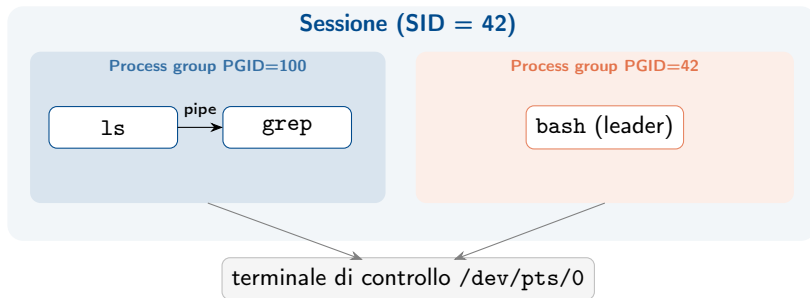
Tutti questi valori sono leggibili in tempo reale da `/proc/[pid]/`:

File	Contenuto
<code>status</code>	PID, PPID, stato, thread
<code>maps</code>	regioni di memoria
<code>fd/</code>	file descriptor aperti
<code>cmdline</code>	comando di avvio
<code>environ</code>	variabili d'ambiente

`/proc` non è un filesystem reale: è una *vista* delle strutture dati del kernel, generata al volo ad ogni lettura. Non occupa spazio su disco.

Gruppi di processo, sessioni e daemon

I processi non vivono isolati: sono organizzati in **process group** e **sessioni**, strutture usate dalla shell per gestire pipeline e job control.



Un **daemon** è un processo che si stacca volontariamente dalla sessione (`setsid()`) per girare in background senza terminale di controllo. `systemd`, `sshd`, `cron` sono tutti daemon.

Thread: condividere lo spazio di indirizzamento

Un **thread** è un flusso di esecuzione che vive *dentro* un processo e ne condivide la memoria. Ogni thread ha il proprio stack, registri e program counter; tutto il resto è in comune.



I thread POSIX si creano con `pthread_create()` e si attendono con `pthread_join()`. Su Linux ogni thread è internamente una `task_struct` separata — lo scheduler li tratta esattamente come i processi.

Processi

- ▶ Isolamento completo della memoria
- ▶ Comunicazione tramite IPC esplicita
- ▶ Context switch più costoso (TLB flush)
- ▶ Un crash non abbatte gli altri
- ▶ Adatti a componenti indipendenti

Esempi: Apache prefork, browser a tab separate, pipeline Unix.

“Threads are a way to create a lighter weight process — a unit of execution with less overhead.”

Thread

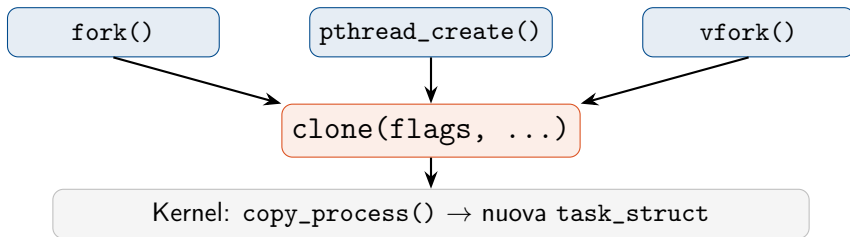
- ▶ Memoria condivisa → comunicazione rapida
- ▶ Creazione e context switch più leggeri
- ▶ Un bug può corrompere l'intero processo
- ▶ Richiedono sincronizzazione esplicita
- ▶ Adatti a lavoro parallelo omogeneo

Esempi: Nginx worker, calcolo parallelo, GUI responsive.

— Kerrisk, *The Linux Programming Interface*, cap. 29

clone(): la primitiva unificante

In Linux, `fork()` e `pthread_create()` sono due modi di chiamare la stessa syscall: `clone()`. La differenza sta nei *flag*: essi decidono **cosa condividere** con il genitore.



clone(): i flag principali

Flag	Effetto
CLONE_VM	condividi la memoria
CLONE_FS	condividi filesystem
CLONE_FILES	condividi fd table
CLONE_SIGHAND	condividi signal handlers
CLONE_THREAD	stesso thread group

fork(): nessuno dei flag sopra → copia tutto.

pthread_create(): tutti i flag sopra → massima condivisione.

I flag CLONE_NEW* — namespace

Flag	Namespace isolato
CLONE_NEWPID	PID tree
CLONE_NEWNET	stack di rete
CLONE_NEWNS	mount points
CLONE_NEWUSER	UID/GID

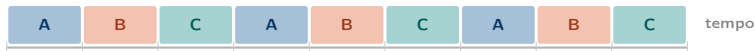
Combinando questi flag si ottiene un **container**: un processo con una vista ristretta del sistema. È esattamente quello che fa Docker.

“Containers are just processes with restricted views of the system, implemented through namespaces and cgroups.”

— Liz Rice, *Container Security*, O'Reilly 2020

Scheduling: chi va per primo?

Lo **scheduler** del kernel decide quale task occupa la CPU in ogni istante. Dal kernel 2.6.23 Linux usa il **CFS** (Completely Fair Scheduler): distribuisce il tempo CPU proporzionalmente al *peso* di ogni task, tenendo un contatore di “tempo virtuale” per ciascuno.



- ▶ **SCHED_OTHER** (CFS) — processi normali; priorità relativa via nice ($-20 \dots +19$)
- ▶ **SCHED_FIFO** — real-time; non prelazionabile
- ▶ **SCHED_RR** — real-time a round-robin
- ▶ **SCHED_DEADLINE** — garanzie temporali esplicite (EDF), introdotto nel kernel 3.14
- ▶ **SCHED_IDLE** — background, solo se CPU è altrimenti libera

Cambiare task non è gratuito. Il kernel deve *salvare* lo stato del task corrente (registri, stack pointer, PC, mappature MMU) e *ripristinare* quello del successivo.



Thread dello stesso processo

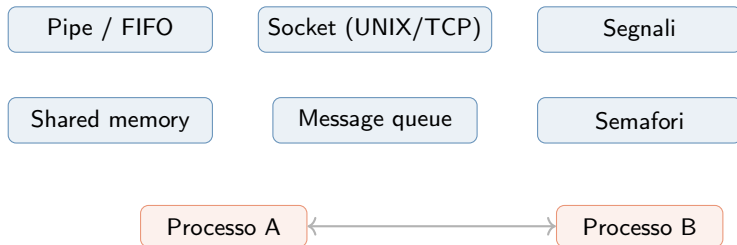
Costo tipico: poche centinaia di ns.

Processi diversi

Il costo può salire a qualche μs

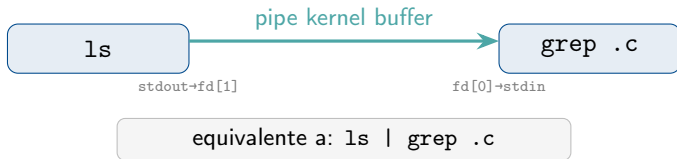
Comunicazione tra processi (IPC)

Poiché i processi non condividono memoria, Linux offre diversi meccanismi per farli comunicare. La scelta dipende dal trade-off tra semplicità, performance e necessità di sincronizzazione.



pipe() e dup2(): connettere i processi

La pipe è il cavo virtuale tra processi. pipe(fd) crea due file descriptor: fd[0] in lettura, fd[1] in scrittura. Con dup2() si reindirizza stdout di un processo su fd[1] e stdin dell'altro su fd[0].



Il pattern in C:

1. pipe(fd) nel padre
2. fork() → due processi, entrambi vedono fd[0] e fd[1]
3. Nel figlio: dup2(fd[1], STDOUT_FILENO), chiudi fd[0], exec()
4. Nel padre: dup2(fd[0], STDIN_FILENO), chiudi fd[1], exec()

Snapshot e monitoraggio

ps aux	snapshot di tutti i processi
pstree	gerarchia padre-figlio
top/htop	monitoraggio in tempo reale
pidstat	statistiche per singolo PID

/proc live

/proc/[pid]/status	stato, PID, PPID
/proc/[pid]/maps	mapping memoria
/proc/[pid]/fd/	fd aperti
/proc/[pid]/cmdline	comando

Tracciamento

strace: mostra le syscall di un processo in tempo reale.

```
strace -p <pid>
```

ltrace: come strace ma per le chiamate a libreria.

perf: profiling hardware (cache miss, branch, IPC).

CPU affinity

taskset lega un processo a specifici core:

```
taskset -c 0,1 ./prog
```

-  Robert Love, *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010.
-  Michael Kerrisk, *The Linux Programming Interface*, No Starch Press, 2010.
-  W. R. Stevens & S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed., Addison-Wesley, 2013.
-  Liz Rice, *Container Security*, O'Reilly, 2020.
-  Linux man-pages, clone(2), <https://man7.org/linux/man-pages/man2/clone.2.html>